Ch 1.Introduction to Java

Features of OOP

Object-Oriented Programming (OOP) is a programming paradigm that organizes software using **objects**. An object represents a real-world entity and contains **data (attributes)** and **methods (functions)** that define its behaviour. Java is an object-oriented programming language that uses OOP concepts to make programs more modular, reusable, and easier to maintain.

OOP Concepts

1.Class

A **class** is a **blueprint or template** used to create objects. It defines the properties (data) and behaviours (methods) that objects of that class will have.

Characteristics of a Class

- It is a user-defined data type.
- It contains data members (attributes) and member functions (methods).
- A class itself does not occupy memory until objects are created.
- Multiple objects can be created from a single class.

Example

A **Car** blueprint contains details such as:

- Color
- Model
- Engine
- Speed

The blueprint is the **class**.

2.Object

An **object** is an **instance of a class**. It is a real entity created from a class and occupies memory.

- Represents a real-world entity.
- Has identity, state, and behavior.
- Occupies memory.
- Can access the methods and properties defined in its class.

Example

If **Car** is a class, then:

- Honda City
- Hyundai Creta
- Tata Nexon

  are individual **objects** of the Car class.

3. Encapsulation
Encapsulation is the process of **combining data and the methods that operate on that data into a single unit (class)** while restricting direct access to the data.
Protects data from unauthorized access.

- Data is accessed through controlled methods.
- Improves security and maintainability.

Example:
A **bank account** hides its balance from direct access. You can only check or modify the balance through authorized operations like deposit or withdrawal.

4. Inheritance

Inheritance is the mechanism by which **one class acquires the properties and behaviors of another class**.

- Promotes code reusability.
- Establishes a parent-child relationship.
- Allows extension of existing functionality.

Example:
A **car** is a type of **vehicle**. The car inherits common features such as wheels and an engine from the vehicle while adding its own features.

5. Polymorphism

Polymorphism means **"one interface, many forms."** The same operation can behave differently depending on the object using it.

- Increases flexibility.
- Allows the same method name to perform different tasks.
- Makes programs easier to extend.

Example:
A person may behave differently in different roles:

- As a **teacher** in a classroom.
- As a **parent** at home.
- As a **customer** in a store.

The same person performs different actions depending on the situation.

6. Abstraction

Abstraction is the process of **hiding unnecessary implementation details and showing only the essential features** to the user.

- Reduces complexity.
- Focuses on what an object does rather than how it does it.
- Improves usability and security.

Example:
When driving a **car**, you use the steering wheel, accelerator, and brakes without needing to understand how the engine or transmission works internally.

Advantages of OOP

- Code reusability
- Data security
- Easy maintenance
- Modularity
- Flexibility
- Scalability
- Reduced code duplication
- Easier debugging and testing

Features of Java

1.Simple
Java is easy to learn and use and debug.
It removes complex and error-prone features of C++, such as explicit pointers and operator overloading.

2.Object-Oriented (OOP)

- Java is based on objects and classes.
- Reusable code.
- It supports the four main OOP principles:
    - Inheritance
    - Polymorphism
    - Encapsulation
    - Abstraction

3. Platform Independent

- Java follows the **"Write Once, Run Anywhere (WORA)"** principle.
- Java source code is compiled into **bytecode**, which can run on any system with a **Java Virtual Machine (JVM)**.

4.Interpreted

Java uses a hybrid execution model. It compiles source code into intermediate bytecode, which the JVM then **interprets line-by-line** into native machine code.

5.Robust

- Java provides strong compile-time and runtime error checking.
- It includes:
    - Automatic Garbage Collection for memory management.
    - Exception Handling to manage runtime errors effectively.

6.Secure

- Java programs run inside a secure **JVM sandbox**.
- It avoids explicit pointers, reducing the risk of unauthorized memory access and improving application security.

7. Portable

- Java bytecode can be executed on different operating systems and hardware platforms without modifying the source code.

8. Multithreading

- Java supports multiple threads, allowing several tasks to run simultaneously.
- This improves CPU utilization and makes applications more responsive.

9. High Performance

- Java uses a **Just-In-Time (JIT) compiler**.
- The JIT compiler converts bytecode into native machine code during execution, improving performance.

10. Dynamic

- Java loads classes, libraries, and methods at runtime as needed.
- It can adapt to changing environments by dynamically linking required modules(new class,libraries,method &object).

11.Open Source

- Java development is supported by the open-source **OpenJDK** community.
- Developers have access to thousands of free libraries and tools.
- Extending functionality and speed up development

12.Interactive

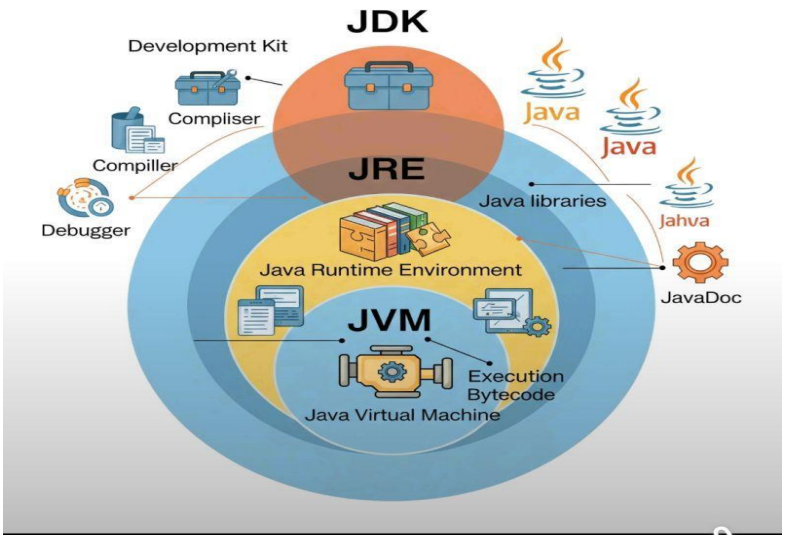
Java features JShell (REPL) for running code snippets instantly without creating full classes, and powerful GUI frameworks (JavaFX/Swing) for highly responsive user applications

History of Java

1991 – Green Project

  - James Gosling, along with Mike Sheridan and Patrick Naughton, began developing the language.
  - The language was originally called **Oak**, named after an oak tree outside Gosling's office.
- **1995 – Renamed to Java**
  - Since the name "Oak" was already trademarked, it was renamed **Java**, reportedly inspired by Java coffee.
  - Sun Microsystems officially released Java on **May 23, 1995**.
  - Java introduced the slogan: **"Write Once, Run Anywhere (WORA)."**
- **1996 – JDK 1.0**
  - The first **Java Development Kit (JDK 1.0)** was released.
  - It included the Java compiler, Java Virtual Machine (JVM), and core libraries.
- **Late 1990s**
  - Java became popular for web applications through **applets**, although applets later became obsolete.
  - It gained widespread use in enterprise software.
- **2004 – Java 5**
  - A major update introduced:
    - Generics
    - Enhanced for-loop
    - Annotations
    - Autoboxing
    - Enumerations (Enums)
- **2006**
  - Most of Java was released as open source under the **OpenJDK** project.
- **2010**
  - Oracle Corporation acquired Sun Microsystems and became the steward of Java.
- **2014 – Java 8**
  - One of the most important releases.
  - Introduced:
    - Lambda expressions
    - Streams API
    - Functional programming features
    - New Date and Time API
- **2017 – Java 9**
  - Introduced the **Java Platform Module System (JPMS)**, also known as Project Jigsaw.
- **2018 onwards**
  - Java adopted a **6-month release cycle**, delivering new features more frequently.

Java Environment



The **Java Environment** is the complete set of software components required to **develop, compile, and run Java applications**. It mainly consists of **JDK (Java Development Kit), JRE (Java Runtime Environment), and JVM (Java Virtual Machine)**.

Components of Java Environment

1. Java Development Kit (JDK)

The **Java Development Kit (JDK)** is a software package used to **develop, compile, debug, and run Java programs**. It is mainly used by Java developers.

Features

- Includes the Java compiler (javac)
- Includes the Java Runtime Environment (JRE)
- Provides development tools such as debugger and documentation generator
- Used for creating Java applications

Main Components

- Java Compiler (javac)
- Java Runtime Environment (JRE)
- Debugger
- Java Documentation Tool (javadoc)
- Java Archiver (jar)

2. Java Runtime Environment (JRE)

The **Java Runtime Environment (JRE)** provides the environment required to **run Java applications**. It does not contain development tools like the compiler.

Run time libraries **java.lang, java.util, java.io and java.nio, java.net**

Features

- Executes Java programs
- Contains JVM and Java class libraries
- Provides runtime support for Java applications
- Intended for users who only need to run Java programs

Components

- Java Virtual Machine (JVM)
- Core Java Libraries
- Supporting files and resources

3. Java Virtual Machine (JVM)

The **Java Virtual Machine (JVM)** is an abstract machine that **executes Java bytecode**. It enables Java programs to run on different operating systems without modification.

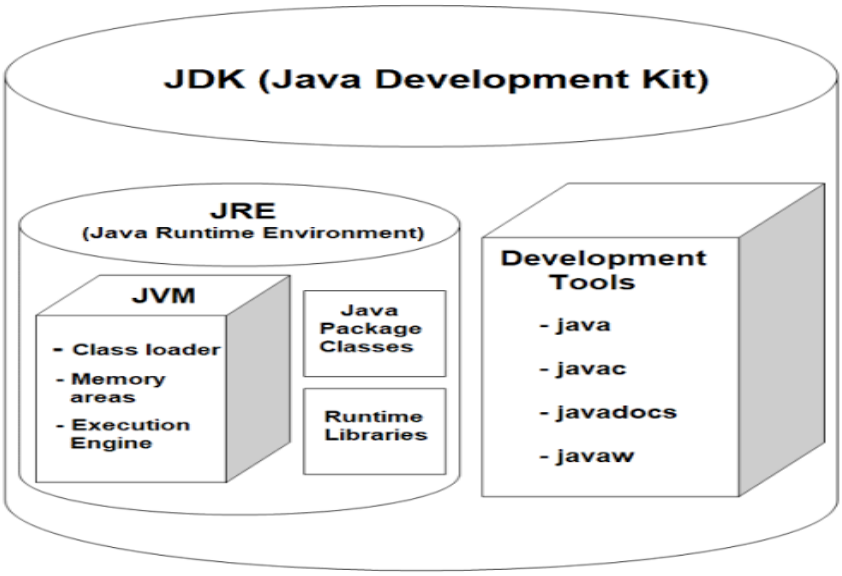
Features

- Converts bytecode into machine code
- Provides platform independence
- Manages memory automatically
- Performs garbage collection
- Ensures security during program execution

Responsibilities

- Loads class files
- Verifies bytecode
- Executes bytecode
- Manages memory
- Handles exceptions
- Performs garbage collection

Java Program Execution Process



Architecture of JVM, JRE & JDK

1. The programmer writes the Java source code (.java file).
2. The **Java Compiler (javac)**, which is part of the JDK, compiles the source code.
3. The compiler generates **bytecode** (.class file).
4. The **JVM** loads and verifies the bytecode.
5. The JVM converts the bytecode into machine code.
6. The operating system executes the machine code and displays the output.

Java API

Java API (Application Programming Interface) is a collection of **predefined classes, interfaces, packages, and methods** provided by Java. It helps developers perform common programming tasks without writing code from scratch.The Java API is also called the **Java Standard Library** because it contains ready-to-use components for building Java applications.

Java API is a set of **predefined packages, classes, interfaces, and methods** that provide reusable functionality for developing Java applications.

Java API Packages

Package	Purpose
java.lang	Basic language classes (String, Math, System, Object, etc.)
java.util	Utility classes such as Collections, Scanner, Date, Random
java.io	Input and Output operations (files, streams)
java.net	Networking and Internet communication

Package	Purpose
java.sql	Database connectivity (JDBC)
java.time	Date and Time API
java.math	Mathematical operations and large numbers
java.awt	GUI components for desktop applications
javax.swing	Advanced GUI components

Java Tools

1. javac (Java Compiler)

javac is the Java compiler that converts **Java source code (.java)** into **bytecode (.class)**.

- Compiles Java programs.
- Detects syntax errors.
- Generates bytecode for the JVM.

To compile Java source files.

2. java (Java Interpriter)

The java command is used to **run Java applications** by executing the compiled bytecode using the JVM.

- Starts the Java Virtual Machine (JVM).
- Executes compiled Java programs.
- Loads required classes automatically.

To execute Java programs.

3. javadoc (Documentation Generator)

javadoc automatically generates **HTML documentation** from comments written in Java source code.

- Creates professional documentation.
- Uses special documentation comments.
- Produces easy-to-read web pages.

To generate project documentation.

Javadoc comments must be placed directly above classes, interfaces, constructors, methods, or fields.

syntax begins with /** ----------

---------- ends with */

Example

```
/**
 * The first sentence acts as a concise summary of the method.
 * Detailed explanations or rules can follow in subsequent lines.
 *
 * @param items The total number of items to process.
 * @return true if processing succeeds, false otherwise.
 */
public boolean processOrders(int items) {
   // Method logic
}
```

Use code with caution.

Tag type define information

Syntax @tag

Javadoc tag always start @ character each tag on new line

No .of tag use to provide the heading

- **@param**: Documents a method or constructor parameter and its expected input.
- **@return**: Describes the data type and meaning of the returned value.
- **@throws** / **@exception**: Specifies exceptions the method might intentionally raise.
- **@author**: Identifies the developer who wrote the source code.
- **@version**: Indicates the current software release or version number.
- **@link**: Creates an inline hyperlink to other parts of your project API
- **@exception**: describe exception thrown by method.

4. jar (Java Archive Tool)

The jar tool is used to **package multiple files and classes into a single archive file (.jar)**.

- Combines many files into one archive.
- Simplifies application distribution.
- Supports compression.

Syntax jar [options] [jar-file] [input-files...]

All files are separated by &,?,*,space.

To package Java applications and libraries.

- **-c**: Create a new archive file.
- **-x**: Extract specified (or all) files from an archive.
- **-t**: List the table of contents for the archive.
- **-u**: Update an existing archive file.
- **-f**: Specify the archive file name as a command-line argument.
- **-m**: Include manifest information from a specified external manifest file.
- **-e**: Specify the application entry point (the class containing the main method).
- **-C**: Temporarily change directories during execution to include files without preserving their absolute folder paths.

Example:

jar cf library.jar DataProcessor.class

is used to **create a Java Archive (JAR) file** containing the compiled Java class DataProcessor.class.

5. jdb (Java Debugger)

jdb is the Java debugger used to find and fix errors **in Java programs.**

- Executes programs step by step.
- Sets breakpoints.
- Examines variables during execution.

To debug Java applications.

6. javap (Java Class File Disassembler)

javap displays information about **compiled Java class files (.class)**.

- Shows methods and fields.
- Displays bytecode instructions.
- Helps understand compiled classes.

Read bytecode(.class) breakdown in to human readable instruction.

To inspect compiled Java class files.

Syntax javap [options] <class_name>

Example
vi Demo.java

```
public class Demo
{
private int id = 101;

public String name = "AI Assistant";
```

```
public void display()
 {
System.out.println("Hello from javap tutorial!");
 }
}
```

javac Demo.java
javap Demo
output

```
 public class Demo
 {
 public java.lang.String name;
 public Demo();
public void display();
}
```

Types of comments

Java supports **three types of comments** that are completely ignored by the compiler.
Increasing Readability of program
Help to user reading code better understand function of program.
All characters available in comment.
1.Single-line //
Single-line comments start with two forward slashes (//).
commentShort notes, single statements, or quick explanations.
e.g

```
int speed = 100; // This is an inline comment explaining the variable
```

2.Multi-line /* comment */
Multi-line comments (or block comments) start with /* and end with */.
Everything placed inside these delimiters can span across multiple lines
Detailed logic explanations or disabling large code blocks.
e.g

```
/* This is a multi-line comment. It is highly useful for explaining complex algorithms or
temporarily disabling chunks of code during debugging. */
 int total = a + b;
```

3.Documentation /** comment */
Writing API specifications to generate external HTML documentation.
Documentation comments are a special category that start with /** and end with */. They must be placed directly above class, interface, or method declarations. You can use special Javadoc tags (like @param and @return) inside them.
e.g

```
/** * Calculates the total area of a rectangle. * * @param width The width of the rectangle.
* @param height The height of the rectangle. * @return The calculated area. */
 public int calculateArea(int width, int height)
{
```

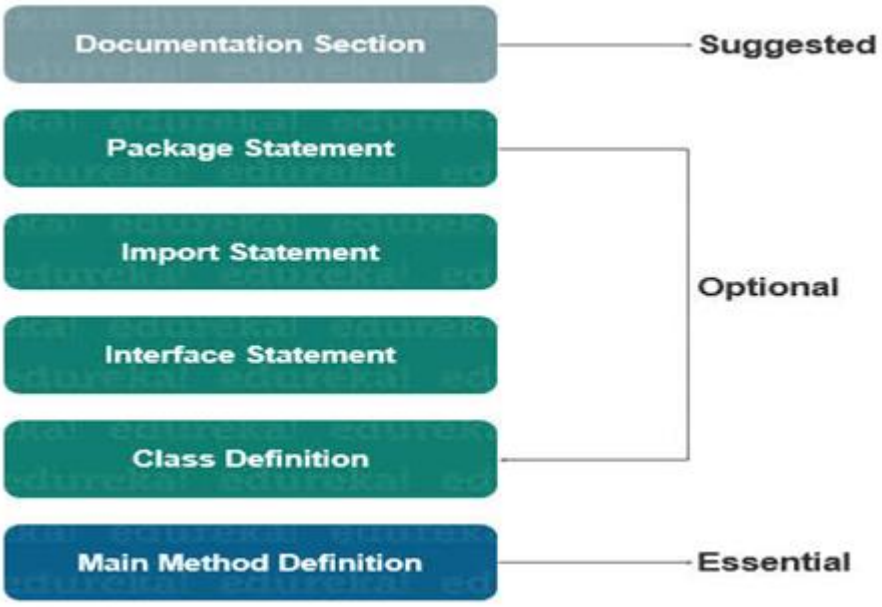
return width * height;
}

Datatype in Java and Array

https://github.com/kalyanikale9/Java/blob/main/datya%20type%20%26loop

https://www.youtube.com/watch?v=Le25I331_yU

Structure of Programming



1.**Open Treminal on Centos**
2.create own folder mkdir folder name
Cd folder name
3.create java file vi filename.java
vi Main.java

4.Press insert
5.Write java code in the editor.
6.Press ESC+Shift+:wq
7.compile java program javac filename.java
javac Main.java

8.run java program java filename
java Main

Example 1

Note: Class Name and program file name should same.

vi Main.java

```
public class Main
{
public static void main(String[] args)
{
// This line prints the message to the screen System.out.println("Hello, World!");
}
 }
```

javac Main.java
java Main
Output
Hello, World!

Simple Java Programming

https://github.com/kalyanikale9/Java/blob/main/Java%20Programming%20-I/Sample%20javaprograms-1.pdf

Accepting Input(Scanner, BufferedReader ,Command Line Arguments)

1. Scanner Class

The Java Scanner Class is part of the java.util package. import java.util. Scanner;
It is the easiest and most preferred method for regular programs because it automatically tokenizes and parses data types.
Read input from console, file, text/string break input into small piece call token Scanner parses primitive data types
Methods: nextInt(), nextDouble(), next(), nextLine()etc.

Syntax Scanner objectName = new Scanner(System.in);

Scanner sc = new Scanner(System.in);

Example

```
import java.util.Scanner;

public class ScannerExample

{

 public static void main(String[] args)

{

// Create Scanner object linked to standard input

 Scanner scanner = new Scanner(System.in);
```

```
System.out.print("Enter your name: ");

 String name = scanner.nextLine();

// Reads a full line of text System.out.print("Enter your age: ");

int age = scanner.nextInt();

 // Reads an integer System.out.println("Hello " + name + "! You are " + age + " years old.");

 scanner.close();

// Close resource

} }
```

2.BufferedReader

The BufferedReader class part of the java.io package. import java.io.BufferedReader;
It stores characters in an internal memory buffer (typically 8KB), making it significantly more performant than Scanner when processing large volumes of text.
However, it only reads strings, meaning you must manually convert types.
Wrap standard input into an InputStreamReader, then into BufferedReader we can read input from command line.

Syntax

Depending on your data source, you will initialize it in one of two ways:

1.Reading from the Console (User Input)

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
```

2.Reading from a File

```
BufferedReader reader = new BufferedReader(new FileReader("filename.txt"));
```

Example

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BufferedReaderExample {
   public static void main(String[] args)
{
      BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

      System.out.print("Enter your city: ");
      String city = br.readLine(); // Reads input as a String

      System.out.print("Enter postal code: ");
      // Manual conversion is mandatory for non-string types
      int code = Integer.parseInt(br.readLine());
```

```
        System.out.println("City: " + city + " | Code: " + code);
    }
}
```

3. Command Line Arguments

Arguments/parameter which supplied to main() from CommandLine at the time of call main() by java interpreter is called CommandLine Arguments.
This arguments are in the string form here store array of string object.
This arguments are passed at the time of running programme (from console to java program input).

Example

vi CommandLineExample.java

```
public class CommandLineExample {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Error: Please provide your name and age as arguments.");
            System.out.println("Example execution: java CommandLineExample John 25");
            return;
        }

        String name = args[0]; // First argument
        int age = Integer.parseInt(args[1]); // Second argument (parsed from String)

        System.out.println("Argument Name: " + name);
        System.out.println("Argument Age: " + age);
    }
}
```

javac CommandLineExample.java
java CommandLineExample ravi 20

Difference Scanner BufferedReader Command Line Arguments

	Scanner	**BufferedReader**	**Command Line Arguments**
Package	java.util	java.io	Built-in via main(String[] args)
Buffer Size	Small (1KB)	Large (8KB)	N/A

Parsing Capabilities	Built-in methods (nextInt(), etc.)	Strings only; manual type-casting needed	Strings only; manual type-casting needed
Thread Safety	Not Thread-Safe	Thread-Safe (Synchronized)	N/A
Exception Handling	Throws unchecked exceptions at runtime	Throws checked IOException	Throws unchecked ArrayIndexOutOfBoundsException

Final Keyword

A **final variable** in Java is a variable whose value **cannot be changed** once it has been initialized. Declaring a variable with the final keyword turns it into a constant, and any attempt to reassign a new value to it will trigger a compile-time error.

Final Keyword use before declaration and initialize it with value.

Syntax final datatype variable = value;

Example final int MAX_SPEED = 120;

```
public class Main
{
 public static void main(String[] args)
{
// Declaring and initializing a final variable
 final int MAX_SPEED = 120;
System.out.println("Maximum Speed: " + MAX_SPEED);
// This line will cause a compilation error:MAX_SPEED = 150;
 } }
```